Lab Manual – 6

Code/Course: 23CSE312/Distributed Systems

Topic: gRPC

[CO05] [BTL-3]
Student Name: _____
Roll No.: _____

Batch/Section: _____
Date: _____

1 Introduction

Modern distributed applications are often composed of multiple services that must communicate efficiently across networks. Traditional communication mechanisms at the browser side employ RESTful APIs, which rely on text-based protocols like HTTP/JSON, which are easy to use but may introduce additional overhead in terms of serialization size and latency. Their capabilities are limited to standard HTTP verbs. To address these limitations, modern systems increasingly adopt high-performance remote communication frameworks. gRPC is an open-source Remote Procedure Call (RPC) framework developed by Google that enables efficient service-to-service communication in distributed systems. It uses HTTP/2 as the transport protocol and Protocol Buffers (Protobuf) as the default data serialization format. gRPC allows a client to invoke methods on a remote server as if they were local procedures, thereby simplifying the development of distributed applications. It also supports features such as bidirectional streaming, multiplexing, authentication, and automatic code generation across multiple programming languages.
.

1.1 Protocol Buffers (Protobuf)

Protocol Buffers, commonly referred to as Protobuf, is a language-neutral, platform-neutral, and extensible mechanism developed by Google for serializing structured data. It serves as both the Interface Definition Language (IDL) and the message format used by gRPC services. In Protobuf, the structure of data exchanged between client and server is defined using a `.proto` file. This file specifies message types and service interfaces using a simple syntax. The Protocol Buffer compiler (`protoc`) then generates source code in the desired programming language, which can be used to serialize and deserialize messages automatically.
Compared to text-based formats such as JSON or XML, Protocol Buffers provide several advantages. They produce smaller message sizes, enable faster serialization and deserialization, and enforce strongly typed communication interfaces. These characteristics make Protobuf particularly suitable for high-performance distributed systems and microservice architectures.

1.2 Advantages of gRPC

gRPC offers several advantages that make it well suited for modern distributed systems and microservices architectures.

1. **High Performance Communication**

gRPC uses HTTP/2 as the underlying transport protocol and Protocol Buffers for data serialization. This combination reduces message size and improves communication efficiency compared to traditional REST-based systems.

2. **Strongly Typed Interface Definitions**
   The use of `.proto` files ensures that service interfaces are clearly defined and type-safe. This minimizes errors during communication between client and server.

3. **Automatic Code Generation**
   gRPC automatically generates client and server stubs in multiple programming languages such as Python, Java, Go, and C++. This simplifies development and reduces boilerplate code.

4. **Support for Streaming Communication**
   Unlike traditional request-response systems, gRPC supports different communication patterns including:

   - Unary RPC
   - Server streaming
   - Client streaming
   - Bidirectional streaming

5. **Cross-Language Interoperability**
   gRPC allows services written in different programming languages to communicate seamlessly with each other, making it suitable for heterogeneous distributed systems.

6. **Efficient Network Utilization**
   HTTP/2 features such as multiplexing and header compression improve bandwidth utilization and reduce communication latency.

2 Aim

To study the working of the gRPC framework and Protocol Buffers, and to develop a client-server application using Python that performs remote procedure calls for arithmetic operations.

3 Objectives

1. To learn how gRPC services are defined using Protocol Buffers.
2. To install and configure the gRPC Python environment.
3. To generate client and server code using the Protocol Buffer compiler (`protoc`).
4. To implement and execute a simple gRPC client-server application.
5. To design and implement a menu-driven calculator service using gRPC.

4 Task A

Refer to the material given below to familiarise and install gRPC for python and try out the sample exercise.
Introduction: `https://grpc.io/docs/what-is-grpc/introduction/`

This page introduces you to gRPC and protocol buffers. gRPC can use protocol buffers as both its Interface Definition Language (IDL) and as its underlying message interchange format.
Core Concepts: `https://grpc.io/docs/what-is-grpc/core-concepts/`
Getting Started in Python:
`https://grpc.io/docs/languages/python/quickstart/`
Windows Installation:

1) Install grpc:
   $ python -m pip install grpcio

2) Install grpc tools (this includes protocol buffer compiler 'protoc')
   $ python -m pip install grpcio-tools

3) Install git if not there: Download link: `https://git-scm.com/download/win`
   Open Git bash in the location where you want grpc folders to be and:

4) Clone the repository to get the example code:
   $ git clone -b v1.62.0 –depth 1 –shallow-submodules https://github.com/grpc/grpc

Try the helloworld sample as described here:
`https://grpc.io/docs/languages/python/quickstart/`
Also update and try with the additional method SayHelloAgain as described

5 Task B – Menu Driven Calculator

- Build a menu-driven calculator application using gRPC.
- Define a gRPC service with methods for basic arithmetic operations (Addition, Subtraction, Multiplication, Division) using Protocol Buffers.
- Create a gRPC server to handle requests and execute the calculator service methods.
- Develop a menu-driven client application allowing users to select operations and input operands interactively.
- Implement error handling, such as managing division by zero scenarios.

Your submission should contain:

1. client and server python codes and the .proto file for part A and B
2. A document with screenshots of output
3. Brief answers to viva questions in notebook